

DEA · Diagnostic Elvis Agent — Installation and Operations Guide

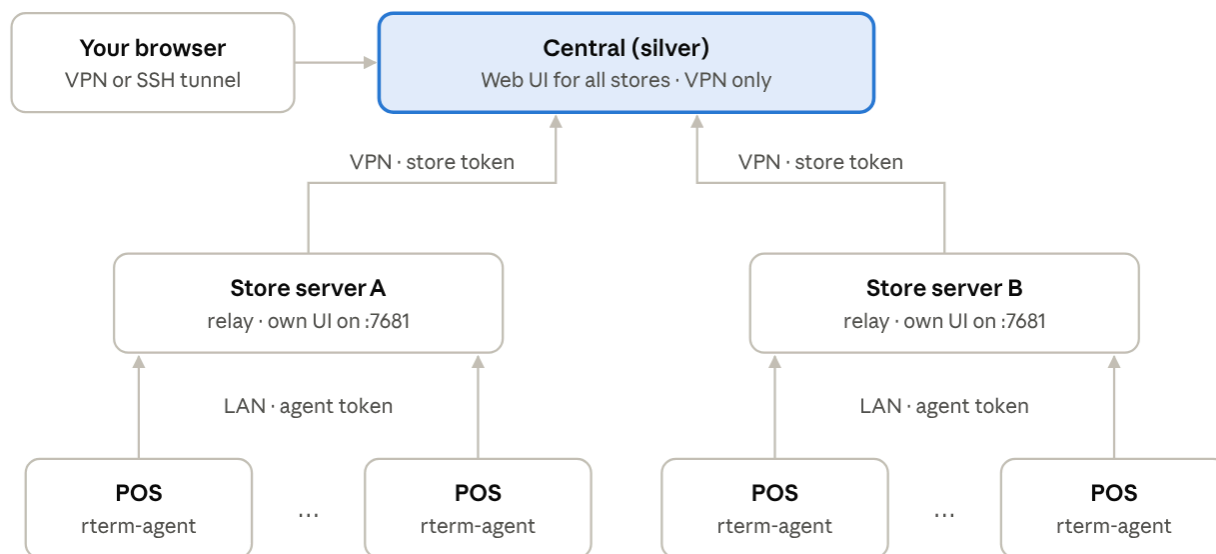
Oct 2, 2026 · @Lito

Overview

DEA (Diagnostic Elvis Agent) gives you browser access to every POS in the fleet from the central server. From one page you get a shell, the POS screen over VNC, log downloads and fleet-wide commands. Every connection is opened outward: each POS dials its store server, and each store server dials central over the VPN. Nothing ever connects into a store or a POS.

- **Central** runs the web UI that you use. It lists every store and, under each store, its POS.
- **Store servers** run the same server in relay mode. Their own POS connect to them, and the store forwards everything up to central over a single link.
- **POS** run a small agent, `dea-agent`. It holds one connection to its store and opens a shell, a VNC bridge or a log archive only when someone asks for one.

Every link is opened from below: POS to store, store to central



Arrows show who opens each connection on port 7681; data then flows both ways.

topology · central, store servers, POS

Central never connects to a store, and a store never connects to a POS. A store or POS behind NAT or a firewall works as long as it can reach the tier above it on port 7681.

Each store server also serves its own UI on port 7681, which shows only that store's POS. This is useful when central is unreachable.

Components

Each tier runs a single piece. `ecli dea install` detects the role automatically: a machine without Docker is a POS, a server with `storeReplicationID` in `ecli.json` is a store, and anything else is central.

Where	What runs	Files and settings
Central	Docker Swarm stack <code>dea</code> , service <code>dea_agent</code> : the binary <code>~/com-elvispos-engine/bin/dea-server</code> (web UI included) run from the <code>openjdk8:stable</code> base image, like <code>bin/Sam.jar</code> ; port 7681 (host mode)	<code>~/com-elvispos-engine/docker-compose.dea-agent.yml</code> (the same file on every server); <code>DEA_*</code> secrets in <code>~/com-elvispos-engine/config/elvisenv</code> ; volume <code>dea_dea-data</code> (list of known POS); POS release in <code>share/RELEASE/elvispos/dea/</code> and <code>share/ALL/elvispos/dea/</code>
Store server	The same stack and binary, linked to the central in <code>REPLICATION_SERVER_ADDRESS</code>	The same compose file and volume; <code>DEA_*</code> secrets in <code>elvisenv</code> ; POS token in <code>share/STORE_SEEDS/dea-agent.env</code> ; binary and POS release copied from central
POS	<code>dea-agent</code> , a detached process (no daemon), running as <code>elvispos</code>	<code>/usr/share/elvispos/dea/</code> : <code>dea-agent</code> , <code>start</code> , <code>stop</code> , <code>dea.env</code> , <code>dea.pid</code> , <code>start.log</code> , <code>log/dea-agent.log</code> ; a line in root's crontab starts it at boot

The server container is attached to `network-backend`, so it can reach `ybservice` at `http://ybservice:7392`. It uses `ybservice` to look up store and device names. On the servers, `ecli start|stop|restart|logs dea` manage the stack like any other ElvisPOS service.

How it works

Each POS and each store server keeps one WebSocket **control link** open to the tier above it. Every terminal, screen or log download then gets its own **session link**, which the lower tier opens on request.

Control links

- A POS connects to `ws://memphisserver:7681` with its store's agent token. It announces its id (hostname, or `--id`), its device id (`ELVIS_DEVICE_ID` from `/usr/share/elvispos/elvisenv`), its IP, uptime and version. If the link drops it reconnects with a 1–30 s backoff, and the `start` loop restarts the agent 5 s after it exits.
- A store server is a server with `STORE_REPLICATION_ID` and `REPLICATION_SERVER_ADDRESS` in its `elvisenv`, the settings ElvisPOS store replication already uses. It connects to `ws://<REPLICATION_SERVER_ADDRESS>:7681` with central's store token, sends `STORE_REPLICATION_ID` as its store id and an up-to-date list of its POS. In central, POS are keyed `<store id>/<pos id>`.

Opening a terminal, screen or log download

1. The browser opens a WebSocket to central (same origin, logged-in cookie).
2. Central sends an `open` request down the control link to the store, and the store passes it to the POS.
3. The POS dials back a new session link to the store, carrying a one-time session id. The store does the same to central.
4. Each server splices the two sockets together, so keystrokes, uploaded files, screen data or the log archive flow straight between the browser and the POS.

The server is never more than a pipe. A shell is a PTY running as `elvispos`. A screen session is a TCP bridge to the POS's own `x11vnc` on `127.0.0.1:5900`. A log download is a `.tar.gz` streamed from `/usr/share/elvispos`.

Commands across POS. *The command bar* sends `sh -c <command>` to each selected POS over its control link and collects exit code and output, with a timeout. A result is accepted only from the link the request went out on.

Resources. Every 60 seconds each POS agent sends its CPU, load, memory and disk usage over its control link. Each server samples its own host on the same schedule, and each store forwards its own figures and those of its POS to central at least every 30 seconds. A server runs in a container, so it measures the host's disks through files that are bind-mounted read-only from them. `eccli dea install` sets these up through `DEA_DISKS` in the compose file.

Names. Each server asks ybservice for `system.store` (store id → description) and `system.devices` (device id → description) through its free-query API. It refreshes every 5 minutes, and immediately when an unknown id appears. The sidebar shows these descriptions, with hostname and IP underneath.

Release. Both programs are single static binaries placed on central: `bin/dea-server` and `share/RELEASE/elvispos/dea/dea-agent`. Store servers copy them from central, and POS copy the agent from their store. See *Releasing a new version*.

Prerequisites

Check each item before installing. Install in order: central first, then each store, then that store's POS.

Need	Where	Why
An <code>ecli</code> build that has the <code>ecli dea</code> command	Central, stores, POS	Every install, upgrade and removal goes through it
<code>bin/dea-server</code> and <code>share/RELEASE/elvispos/dea/dea-agent</code>	Central	The server binary and the POS agent; stores and POS copy them from there
Docker Swarm active, <code>openjdk8:stable</code> image, <code>network-backend</code> and <code>ybservice</code> running	Central, stores	The stack runs the binary in the same base image as the Java services and reads names from <code>ybservice</code>
Store replication set up: <code>STORE_REPLICATION_ID</code> and <code>REPLICATION_SERVER_ADDRESS</code> in <code>config/elvisenv</code>	Stores	The store id shown on central, and the central the store links to (<code>ecli configure --set-store-replication <store id></code> sets both)
VPN between stores and central	Stores → central	Stores dial port 7681 on <code>REPLICATION_SERVER_ADDRESS</code> , and central accepts only VPN addresses
Port 7681/tcp reachable	Central from stores (VPN); store from its POS (LAN)	The only port the system uses
<code>ecli</code> SSH link from the store to central (<code>sshParms</code> in <code>ecli.json</code>)	Stores	Store install reads central's store token and copies the binaries over it
<code>rsync</code> access to <code>memphisserver::share</code>	POS	The POS gets its agent, scripts and token from its store

Need	Where	Why
<code>elvispos</code> account with <code>sudo</code> ; <code>x11vnc</code> on <code>127.0.0.1:5900</code> (optional, for <i>Screen</i>)	POS	The agent and its shells run as <code>elvispos</code> ; <code>sudo</code> is used only to write root's crontab

Install 1: central server

Installing central is one command, and running it again later is safe:

```
ecli dea install
```

It does the following:

1. Writes the `DEA_*` secrets into `config/elvisenv`. Values that already exist are kept, so they survive re-runs. It generates the agent token, the admin password (user `admin`), the session secret and the **store token**.
2. Sets `DEA_ALLOW_CIDRS` to the VPN subnets of this host's `tun*` / `wg*` interfaces, plus `127.0.0.0/8` and the `docker_gwbridge` gateway (for example `7.7.7.0/24, 127.0.0.0/8, 172.18.0.1/32`). Any other address gets `403`. To override it, use `--allow-cidrs`.
3. Writes the POS agent's `start` / `stop` scripts into `share/ALL/elvispos/dea/`, next to the agent in `share/RELEASE/elvispos/dea/`.
4. Writes `docker-compose.dea-agent.yml`, deploys the stack `dea` with `bin/dea-server`, restarts it so a new binary is picked up, and waits until it is listening.

When it finishes, open `http://<central VPN IP>:7681` from the VPN and log in as `admin`. The password is `DEA_ADMIN_PASSWORD` in `~/com-elvispos-engine/config/elvisenv`. From outside the VPN, use an SSH tunnel and open `http://localhost:7681`:

```
ssh -p 4168 -L 7681:localhost:7681 elvispos@<central host>
```

`bin/dea-server` must be in place first; see *Releasing a new version*.

Install 2: each store server

On each store server, run the same command after central is installed:

```
ecli dea install
```

It does the following:

1. Checks that store replication is set up: `STORE_REPLICATION_ID` (the store id shown on central) and `REPLICATION_SERVER_ADDRESS` (the central to link to, on port 7681) in `config/elvisenv`.
2. Reads central's store token over the eclissh link into `DEA_UPSTREAM_TOKEN`. You can pass it yourself with `--store-token`.
3. Generates this store's own agent token, admin password and session secret.
4. Copies `bin/dea-server`, `share/RELEASE/elvispos/dea/` and `share/ALL/elvispos/dea/` from central.
5. Publishes the POS token in `share/STORE_SEEDS/dea-agent.env`, where the store's POS fetch it.
6. Deploys the stack and waits until central confirms the link (*Linked to central*).

The store now appears on central's *Fleet* page with a STORE badge. Each store has its own agent token, so a POS token from one store does not work on another. `config/elvisenv` and `share/STORE_SEEDS` are not synced between servers.

Install 3: each POS

On each POS, as `elvispos`, run the same command again:

```
ecli dea install          # name in the UI = hostname
ecli dea install --id cassa3 # or a chosen name
```

Without store servers. In a setup with only central and POS, POS install the same way. The server in the POS's `ecli.json` is then central, which publishes the agent, its scripts and the POS token just as a store does. Its POS appear in the *Fleet* tree directly under the central server. Central accepts them only from `DEA_ALLOW_CIDRS`, so POS on a network outside the VPN need their subnet added with `ecli dea install --allow-cidrs ...` on central.

It does the following:

1. Fetches the agent token from `memphisserver::share/STORE_SEEDS/dea-agent.env` by rsync. The token is never printed or logged.
2. Fetches `dea-agent` from `share/RELEASE/elvispos/dea/` and `start / stop` from `share/ALL/elvispos/dea/` into `/usr/share/elvispos/dea/`.
3. Writes `dea.env` (mode 600) with `DEA_SERVER=ws://memphisserver:7681`, `DEA_TOKEN`, and `DEA_ID` if `--id` was given.
4. Adds this line to root's crontab, next to `startvnc` and `startall`:

```
@reboot su elvispos -c /usr/share/elvispos/dea/start >>
/usr/share/elvispos/dea/start.log 2>&1
```

5. Restarts the agent in the background and waits for its pid. It is safe to run this from a DEA shell on that same POS: the shell drops for a moment, and the POS reconnects.

`start` runs the agent in its own session (`setsid`) inside a restart loop and records the loop's pid in `dea.pid`. It is not run in `screen`, because the POS `startall` runs `killall -9 screen` at every boot. `stop` ends the whole process group. The agent log is `log/dea-agent.log` (mode 600, directory 700), and the previous run's log is kept as `dea-agent.log.1`.

Releasing a new version

A new version is two files copied into place, from v2.4.0: no `ecli dea install` is needed. Each program checks its own binary every 30 seconds; when the file has been replaced, it runs the new one with `-version`, waits until no terminal or screen is open (at most one hour), and restarts into it. Docker's restart policy (servers) or the start loop (POS) starts the new file. A file that does not run is logged and ignored. `ecli dea install` is needed once per machine, and again only when a new `ecli` changes the setup.

1. **Build** in `~/dev/remote-terminal` (Node 22 for the UI):

```
make dist VERSION=vX.Y.Z
```

This writes two static linux/amd64 binaries to `dist/`: `dea-server` (server and web UI) and `dea-agent` (POS). Check them with `dist/dea-server -version` and `dist/dea-agent -version`.

2. **Central:** copy `dist/dea-server` to `~/com-elvispos-engine/bin/dea-server` and `dist/dea-agent` to `~/com-elvispos-engine/share/RELEASE/elvispos/dea/dea-agent`. Within a minute the log shows `self-update: vX.Y.Z found` and then `dea-server vX.Y.Z listening`. `ecli restart dea` switches at once.
3. **Store servers:** the regular server sync copies `bin/dea-server` (with the jars) and the POS release from central; each store then restarts into the new server by itself.
4. **POS:** the agent arrives with the normal POS sync of `share/RELEASE` (when `swdist.device_release` triggers it, or at once with `ecli sync --force`), from the store or from central when the POS connects to it directly. The agent restarts into it by itself; its log shows `dea-agent vX.Y.Z`.

Servers self-update because their compose file sets `DEA_SELF_UPDATE=1` and mounts the whole `bin/` folder: a single-file mount would keep showing the old binary. The agent self-updates unless its `dea.env` holds `DEA_SELF_UPDATE=0`. A reinstall keeps the POS name

(`DEA_ID`) unless `--id` gives a new one.

To see which agent version a POS runs, open its row; *Details* shows *Agent*.

Using the web UI

The home page, **Fleet**, shows the whole fleet as one tree: the central server first, then each store server with its POS indented beneath it. Every row carries live CPU, memory, disk, load and uptime figures. Clicking a row opens that device in its own tab, and terminals, screens and logs open in further tabs. The look follows the *ElvisPOS Remote Terminal* design system.

Header bar. The brand comes first, then the tabs: *Fleet* is always first, followed by each device, terminal and logs tab you open. On the right are the light/dark switch, the language (English or Italiano), and *Sign out*. While a terminal tab is active, its split buttons (1, 2 or 4 panes) and *Type in all* also appear in the bar. Sign in as `admin` ; a login lasts 12 hours, and the browser remembers the theme and language.

Summary tiles. The tiles count stores up, POS online, offline devices, devices at 80–89% or with a late report, and devices at 90% or more. Clicking a tile applies the matching filter. A dot on the *Fleet* tab turns amber or red when anything needs attention.

Toolbar.

- The search box matches store name, POS name, hostname, IP, store ID and POS ID. A matching POS always shows under its store and the central server.
- The filters are *All*, *Online*, *Offline* and *Needs attention*. *Needs attention* shows warnings, late reports and offline devices.
- *Sort*: **A–Z** orders stores and POS by name; **Store ID** orders stores by number and POS by device ID.
- *Expand all / Collapse all* folds the stores. A collapsed store still shows `! n` when n of its POS need attention.
- *Add store or POS* opens the install instructions: the store settings block on central, `eccli dea install` for a POS, and on a store server, the central it is linked to.

The list. Each row has the same columns: select · device · CPU · memory · disk (the fullest mount) · load (1 minute) · uptime · reported · shortcuts · ... The shortcuts open the device's **Overview**, its **terminal and screen**, and its **Logs**; store and central rows have Overview only, and an offline POS has only Overview available.

- The device line shows the name (the `system.store` or `system.devices` description), a CENTRAL or STORE badge, and on stores the POS online/known count. Under it come hostname · IP · POS ID.

- Meters are green under 80%, amber at 80–89% and red at 90% or more, and always show the number. Hover over the memory meter to see GB used and swap, and over the disk meter to see every mount.
- A report older than 2½ minutes turns the dot and the *Reported* time amber. Offline devices keep their place with a hollow dot and *Offline · last seen* A POS whose agent is older than v1.6.0 shows a note to update it.
- Keyboard: ↑/↓ move between rows, Enter opens a row, →/← expand or collapse a store, and Space ticks a POS.
- The ⋮ menu offers *Open terminal*, *Screen*, *Logs* and *Overview*, and *Remove from list* for an offline device. A removed device comes back if it connects again.

Running a command. Tick POS rows, or a store's box to tick all its POS. A command bar then floats at the bottom of the page with the command, a timeout and *Run*. The command runs once on each ticked POS as `sh -c`, and *Command results* lists the exit code and output of each, failures first.

Overview. Clicking a row, or its Overview shortcut, opens the device in an OVERVIEW tab. The page has breadcrumbs back up the tree (*Fleet › Central server › store › POS*), cards for CPU, memory, load and uptime, a table of every disk with used, free and size, and the device details. A POS overview adds *Open terminal*, *Screen*, *Logs* and a *Last command* box for running a command on that POS alone. A store overview also lists its POS with the same rows. For an offline device, the page shows its last report.

Terminals. Opening a POS starts a tab, badged TERMINAL + SCREEN, split in two: the terminal on the left and the POS screen on the right.

- The split buttons in the header bar set 1, 2 or 4 panes; removing a pane leaves 3, with one on the left and two stacked on the right.
- Drag the line between panes to resize them; double-click it to split evenly again.
- The × in a pane's header closes that view and removes the pane, and the others grow to fill the space; the last pane's × closes the tab.
- A tab holds POS of **one store**: its empty panes offer only that store's POS, and opening a POS of another store starts a new tab.
- *Type in all* (amber when on) sends your typing to every terminal pane of the tab. If a shell drops, press Enter in its pane to reconnect.
- Every pane header has a *TERMINAL / SCREEN* switch. If another pane already shows the other view of that POS, the two panes swap; otherwise the other view stays connected in the background.
- Drop files on a terminal pane, or click **Upload** in its header, to save them on the POS in the folder the terminal is in: see [Uploading files to a POS](#).

Screen (VNC). The page first checks that VNC is running on the POS. Type the POS's VNC password in the browser; it is never stored on the servers. The screen opens **view only**. *Take control* passes your mouse and keyboard to the POS, and *Ctrl+Alt+Del* is available only while you are in control. Close GUI, next to it, closes the cash register program (Chrome) on the POS after a confirmation: Chrome is asked to quit and forced after 5 seconds, and the pane says GUI closed, The GUI was not running, or why it could not. It runs as a logged command, like the command bar, so it works even while the screen is view only.

Logs. The Logs tab lists the log files under `/usr/share/elvispos` with size and age: `*.log*` files, or files in `log / logs` folders, up to 3 levels deep. Use *Download selected* or *Download all*; the POS compresses the files and you receive one `.tar.gz`.

Uploading files to a POS

A file dropped on a terminal pane is saved on that POS, in the folder the terminal is in at that moment. If you ran `cd /usr/share/elvispos/gui`, the file lands there; nothing else to choose. It needs `dea-agent` v2.3.0 or later on the POS.

How to upload

1. Open the POS terminal and `cd` to the folder that should receive the file.
2. Drag one or more files from your desktop onto the terminal pane. The pane shows *Drop to upload into the terminal's current folder*. Or click **Upload** in the pane header and pick the files.
3. A status line at the bottom right of the pane shows *Uploading price.csv · 45% · 1.0 MB of 2.3 MB*, with **Cancel**.
4. When the POS has the file in place it reads *price.csv uploaded to /usr/share/elvispos/gui (2.3 MB)*. Several files go one after the other (2 of 3). Type `ls` to see them.

What DEA checks

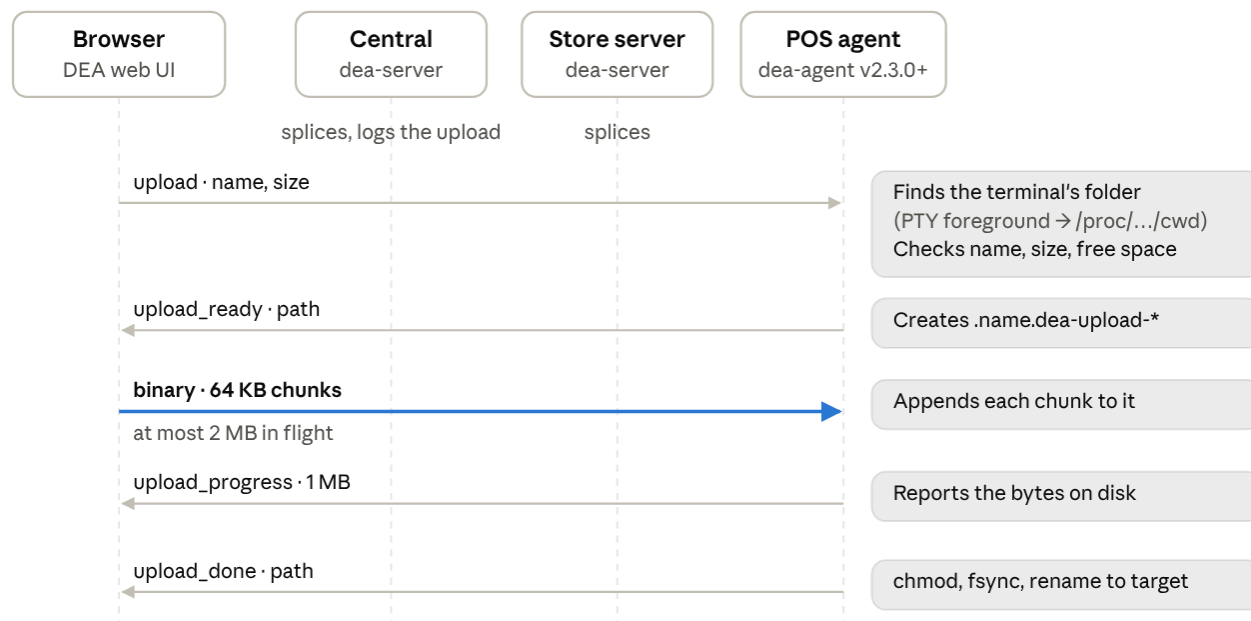
Case	What happens
A file with that name is already there	A dialog asks <i>Replace</i> or <i>Keep the old one</i> . A replaced file keeps its permissions, so a script stays executable. The old file is kept in a hidden folder next to it, by day: <code>.dea-replaced/2026-10-06/price.csv</code> (a second replace that day adds the time, <code>price.csv.143940</code>). If the old file cannot be kept, it is not replaced.

Case	What happens
A folder is dropped	Refused: <i>zip it and upload the archive.</i>
The file is larger than 1 GB	Refused before anything is sent.
Less free space in the folder than the file plus 16 MB	Refused: <i>not enough free space in ...</i>
The folder is not writable by <code>elvispos</code> (e.g. <code>/usr/bin</code>)	Refused: <i>no permission to write in ...</i> Upload to <code>/tmp</code> , then <code>sudo mv</code> it.
The terminal runs <code>sudo -i</code> or <code>su</code> , so its folder belongs to root	Refused: the agent cannot see that folder. Leave the root shell, or upload to <code>/tmp</code> and move the file.
The POS agent is older than v2.3.0	<i>This POS agent cannot receive files:</i> update it with <code>ecli dea install</code> on the POS. Nothing is sent.
The terminal is disconnected	<i>The terminal is not connected:</i> press Enter in the pane to reconnect, then drop again.

Files are written as `elvispos` with mode `644`. Only terminal panes accept files; a SCREEN pane ignores them, and a file dropped elsewhere on the page is ignored too, so the browser never leaves DEA to open it.

How an upload travels

A dropped file rides the terminal link straight to the POS agent



Name taken: upload_error with exists → the browser asks Replace → upload again with overwrite

Cancel, failure or a closed terminal: the agent deletes the temporary file; the target is never half-written

file upload · browser → central → store → POS agent

Central and the store only pass the frames along; the agent decides where the file goes and answers every step, so the browser shows what is really on the POS disk.

How an upload works, step by step

The file travels inside the terminal's own session link, the same chain of WebSockets that carries your keystrokes (see *How it works*). Central and the store splice that link unchanged, so only the browser and the POS agent take part, and an upload works on any store server version. On the session link, text frames carry JSON messages and binary frames carry bytes; while an upload is open, every binary frame from the browser is file data, never keystrokes.

- 1. Ask.** The browser sends `{"type": "upload", "name": "price.csv", "size": 2411724}` and waits up to 10 seconds for the answer.
- 2. Find the folder.** The agent owns the terminal's PTY. It asks the PTY for its foreground process group (`TIOCGPGRP`), which is the shell, or the program the shell is running, and reads that process's current folder from `/proc/<pid>/cwd`.
- 3. Check.** The agent refuses a bad name (empty, `.`, `..`, containing `/`, over 255 characters), a file over 1 GB, a target that is a folder, or too little free space. If the name exists and the request did not say `overwrite`, it answers `upload_error` with `exists`:

`true` and the full path; the browser shows *Replace / Keep the old one* and, on Replace, asks again with `"overwrite": true`.

4. **Open a temporary file.** The agent creates `.price.csv.dea-upload-<random>` in the target folder and answers `{"type": "upload_ready", "path": "/usr/share/elvispos/gui/price.csv"}`.
5. **Send the bytes.** The browser reads the file in 64 KB chunks and sends each as a binary frame. It keeps at most 2 MB queued on the socket, so a slow VPN link slows the upload instead of filling the browser's memory. Every 1 MB written, the agent answers `upload_progress` with the bytes saved so far; the percentage on screen comes from these, so it shows what is really on the POS disk.
6. **Put it in place.** After exactly `size` bytes the agent gives the file its mode (`644` , or the old file's mode when replacing), flushes it to disk (`fsync`), and renames it over the target in one step. It then answers `{"type": "upload_done", "path": "...", "size": ...}` and the browser shows the green status line.
7. **Cancel or failure.** *Cancel* sends `upload_cancel` ; the agent deletes the temporary file. If writing fails (disk full), the agent answers `upload_error` , the browser stops and cancels, and anything still in flight is read and dropped. If the terminal closes mid-upload, the agent deletes the temporary file too. The target file is never left half-written: it changes only at the final rename.

Safety and audit

- **Older agents receive nothing.** An agent before v2.3.0 ignores the `upload` message, so the browser never gets `upload_ready` , sends no bytes, and after 10 seconds shows *update the agent*. File bytes can never be typed into an old agent's shell.
- **Typing waits during an upload.** While a file goes out, the pane drops keystrokes (and *Type in all* skips that pane), so file data and typing never mix. Typing works again as soon as the status line turns green or red.
- **Same rights as the terminal.** The agent writes as the account its shells run as, `elvispos` . An upload cannot reach a folder that `elvispos` cannot write from the shell, and needs no new token or port.
- **Audit.** Every upload is recorded on the server and on the POS, from the request to its outcome, with who sent it: see *Upload audit* below.
- **Limits.** 1 GB per file; one upload at a time per terminal; folders must be zipped first. Speed follows the VPN link to the store; the agent itself saved a 40 MB file in under half a second.

Upload audit

Every upload leaves a record on both ends, from v2.3.1: the server the user signed in to records who sent what and how it ended, and the POS records what it wrote, with the SHA-256 of the saved file. Each event is one line in the service log and one JSON line in a dedicated `uploads.log`.

Where	Files	Who is named
Central (or the server the user signed in to)	<code>~/com-elvispos-engine/logs/dea/dea-server.log</code> and <code>uploads.log</code> (10 MB, 20 old files kept)	The DEA user and the browser address
Store server in between	The same two files on the store	The DEA user and address sent by central (<code>from: upstream</code>)
POS agent	<code>/usr/share/elvispos/dea/log/dea-agent.log</code> and <code>uploads.log</code> (10 MB, 10 old files kept)	<code>by: admin@10.8.0.5</code> , sent by central when the terminal opens

Event	Logged when
requested	The browser asks to upload a file (server only)
exists	The name is taken; the user is asked to replace it
refused	The POS refuses it, with the reason (permission, space, size, root shell, bad name, another upload running)
started	The POS has opened the temporary file; the target path is known
saved	The file is in place; carries its SHA-256 and, when it replaced a file, replaced: true with the old file's size, its SHA-256 and backup, the path where it was kept
failed	Writing failed after the start (disk full, rename failed)
cancelled	The user pressed Cancel; carries the bytes sent so far
interrupted	The terminal closed during the upload; carries the bytes sent so far

Each JSON line holds: time, event, server, POS, user and address (by on the POS), session id, file name, size, whether overwrite was asked, final path, bytes sent, SHA-256, replaced, old size, old SHA-256 and backup path, duration in ms, and the error. An example saved line on central (values illustrative):

```
{
  "time": "2026-10-06T11:57:15+02:00",
  "event": "saved",
  "server": "amacrai-central-server",
  "agent": "12/memphis-pos-169",
  "user": "admin",
  "from": "10.8.0.5",
  "session": "4d87a2c0",
  "name": "price.csv",
  "size": 2411724,
  "overwrite": true,
  "path": "/usr/share/elvispos/gui/price.csv",
  "bytes": 2411724,
  "sha256": "27fe6057cf960dbc3c2ad1217f40ab7edbf5db991e431590f0c518b156bf6302",
  "replaced": true,
  "old_size": 2398110,
  "duration_ms": 1210
}
```

To list a day's uploads on central: `grep '"event": "saved"' ~/com-elvispos-engine/logs/dea/uploads.log | grep 2026-10-06`.

The agent never puts a file in place without writing its `saved` line and SHA-256. The server's record ends in `saved` too, or in `interrupted` when the browser left before the POS confirmed; the POS record then tells whether the file was saved.

Operations

All day-to-day tasks go through `ecli`. Run them on the machine concerned.

Task	Command	Where
Check state	<code>ecli dea status</code>	Servers: role, store id and central, allowed networks, service state. POS: running pid, boot entry, last log lines
Start / stop / restart the server	<code>ecli start dea</code> , <code>ecli stop dea</code> , <code>ecli restart dea</code>	Central, stores
Server log and audit trail	<code>ecli logs dea</code> or <code>docker service logs dea_agent</code>	Central, stores
Start / stop the agent	<code>/usr/share/elvispos/dea/start</code> , <code>.../stop</code> (as <code>elvispos</code>)	POS
Agent log	<code>/usr/share/elvispos/dea/log/dea-agent.log</code> ; boot output in <code>start.log</code>	POS
Remove	<code>ecli dea uninstall</code> (<code>-f</code> skips the confirmation)	Any
Remove with settings and data	<code>ecli dea uninstall --purge</code>	Central, stores

On a POS, `uninstall` stops the agent, removes the crontab line and deletes `/usr/share/elvispos/dea`. On a server, it removes the stack. `--purge` also deletes the `DEA_*` keys from `elvisenv` and the `dea_dea-data` volume.

Rotating secrets. Delete the key from `config/elvisenv` and run `ecli dea install` again. A new value is generated.

- `DEA_ADMIN_PASSWORD`: takes effect immediately.
- `DEA_AGENT_TOKEN` on a store: re-run `ecli dea install` on each of that store's POS.
- `DEA_STORE_TOKEN` on central: re-run `ecli dea install` on every store.

Settings. DEA keeps only its secrets in `~/com-elvispos-engine/config/elvisenv`, all written by `ecli dea install`. Everything else comes from ElvisPOS itself: names from `ybservice` on `network-backend`, and on a store the store id and central from the replication settings.

Key	Tier	Meaning
DEA_ADMIN_USER , DEA_ADMIN_PASSWORD	Central, stores	Web UI login (user <code>admin</code>)
DEA_SESSION_SECRET	Central, stores	Signs login cookies
DEA_AGENT_TOKEN	Central, stores	Secret this server's POS present (published to POS via <code>share/STORE_SEEDS</code>)
DEA_STORE_TOKEN	Central	Secret store servers present when they link
DEA_ALLOW_CIDRS	Central (optional on stores)	Networks allowed to connect; others get 403
DEA_UPSTREAM_TOKEN	Store	Central's <code>DEA_STORE_TOKEN</code>
STORE_REPLICATION_ID	Store (ElvisPOS setting)	Store id shown on central
REPLICATION_SERVER_ADDRESS	Store (ElvisPOS setting)	Central to link to; DEA adds port 7681

On the POS, `/usr/share/elvispos/dea/dea.env` holds `DEA_SERVER` , `DEA_TOKEN` and the optional `DEA_ID` .

Security

Access is gated three ways: central is reachable only from the VPN, each tier has its own secret, and nothing on a store or POS accepts inbound connections from central.

- **Network.** Central answers only addresses in `DEA_ALLOW_CIDRS` (VPN, localhost, Docker gateway). From outside, use the SSH tunnel. Traffic is plain `ws` / `http` and relies on the VPN for encryption.
- **Three separate secrets.** The admin login (per server), the POS agent token (per store) and central's store token. A leaked POS token gives no access to the UI or to other stores.
- **Browser.** Login cookies are HMAC-signed and expire after 12 hours. WebSocket requests from another origin are rejected.
- **POS.** Shells run as `elvispos` , not root. `dea.env` and the agent log are readable only by `elvispos` . Log downloads are limited to regular files under `/usr/share/elvispos` (no symlinks). The VNC password never leaves the browser and the POS.

- **Audit.** Each server logs logins, commands run, terminal, screen and log-download sessions with who and which POS. Read them with `docker service logs dea_agent`.

Troubleshooting

Symptom	Cause	Fix
Central page returns <code>403 Forbidden</code>	Your address is not in <code>DEA_ALLOW_CIDRS</code>	Connect to the VPN or use the SSH tunnel; extend with <code>ecli dea install --allow-cidrs ...</code>
Install: <i>bin/dea-server not found</i>	The binary is not on central (or not copied to the store yet)	Put it in central's <code>bin/</code> (<code>make deploy</code>), then run the install again
Store install: <i>STORE_REPLICATION_ID and REPLICATION_SERVER_ADDRESS must be set</i>	Store replication not set up	<code>ecli configure --set-store-replication <store id></code>
Store install: <i>Not linked to central yet</i>	Port 7681 blocked on the VPN, wrong <code>REPLICATION_SERVER_ADDRESS</code> or token	<code>ecli dea status</code> and <code>ecli logs dea</code> on the store; re-run <code>ecli dea install</code> on the store
POS install: <i>Agent token not found</i>	Store not installed, or rsync to <code>memphisserver</code> fails	Install the store first; check <code>rsync elvispos@memphisserver::share/</code> from the POS
POS install: <i>dea is not published on memphisserver yet</i>	The store has no POS release	Put the agent on central, run <code>ecli dea install</code> on central, then on the store
POS offline after reboot	Crontab line missing, or agent cannot reach the store	<code>ecli dea status</code> on the POS; read <code>start.log</code> and <code>log/dea-agent.log</code>
Two POS overwrite each other in the list	Same hostname in the same store	<code>ecli dea install --id <unique name></code> on one of them
Screen: <i>VNC is not running on this POS</i>	x11vnc not listening on <code>127.0.0.1:5900</code>	Start VNC on the POS (<code>startvnc</code>)
Screen: <i>Wrong VNC password</i>	Password typed does not match the POS	Use that POS's VNC password
Names show ids instead of descriptions	ybservice unreachable or the id is missing from <code>system.store / system.devices</code>	Check that ybservice runs; names refresh within 5 minutes
Upload: <i>this POS agent cannot receive files</i>	The POS runs dea-agent older than v2.3.0	Release the new agent on central and the store, then run <code>ecli dea install</code> on the POS

Symptom	Cause	Fix
Upload: <i>no permission to write in ...</i>	The terminal's folder is not writable by <code>elvispos</code>	<code>cd /tmp</code> , upload, then <code>sudo mv</code> the file where it belongs
Upload: <i>the terminal runs a program as another user</i>	The pane is in a root shell (<code>sudo -i</code> , <code>su</code>), whose folder the agent cannot see	Leave the root shell with <code>exit</code> , or upload to <code>/tmp</code> and move the file as root